Statistical and Computational Models

Lecture 17: Splines and GAMs

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Roadmap

Flexible parametric models

Additive models

Comparing models

Our classification problem

Previously we estimated $\pi(\boldsymbol{x}) = \mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$ in two ways:

$$\begin{aligned} \text{Logit:} \quad & \operatorname{logit} \pi(\boldsymbol{x}) = \boldsymbol{x}^\top \beta \\ \text{Tree:} \quad & \hat{\pi}(\boldsymbol{x}) = \hat{p}_m \text{ for } \boldsymbol{x} \text{ in leaf } m \end{aligned}$$

The logit is parametric: we assume a functional form, and estimate the $\hat{\beta}$s.

A tree is non-parametric: we assume no form, and thus estimate no $\hat{\beta}$s.

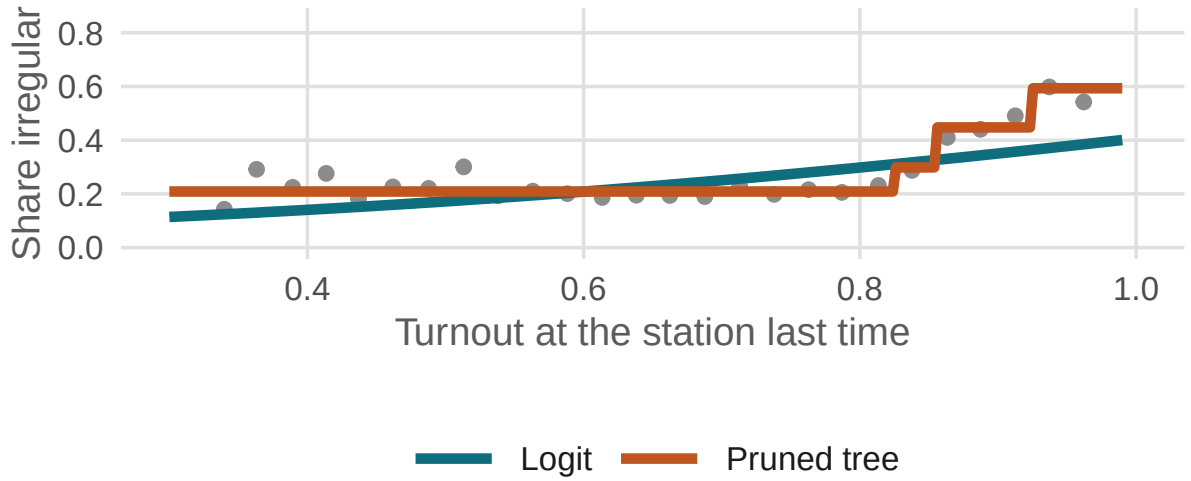
Now we reconsider **parametric** models, but allow the data to decide how flexible our model (logit in this case) should be.

This is particularly useful if we expand our goals:

1. Rank this election's stations.
2. Learn how the chance of an irregularity changes with each feature.

Turnout and irregularities

Let's start with a single feature, turnout in the previous election:



Binning units by prior turnout, we see a clearly non-linear relationship with our dependent variable.

The logit gives a smooth $\hat{\pi}$, but its shape comes from our modeling choices.

The tree captures the non-linearity, but $\hat{\pi}$ is no longer smooth, and the relationship between x and Y is thus hard to interpret.

Basis functions

To give the logit more flexibility, we use a trick you are already familiar with:

We model Y not as a function of x, but as a function of $b_j(x)$, where $b_j(\cdot)$ is a **basis function** that generates new representations of x out of x:

$$\text{logit}\,\pi(x) = \beta_0 + \sum_{j=1}^{J} \beta_j\, b_j(x)$$

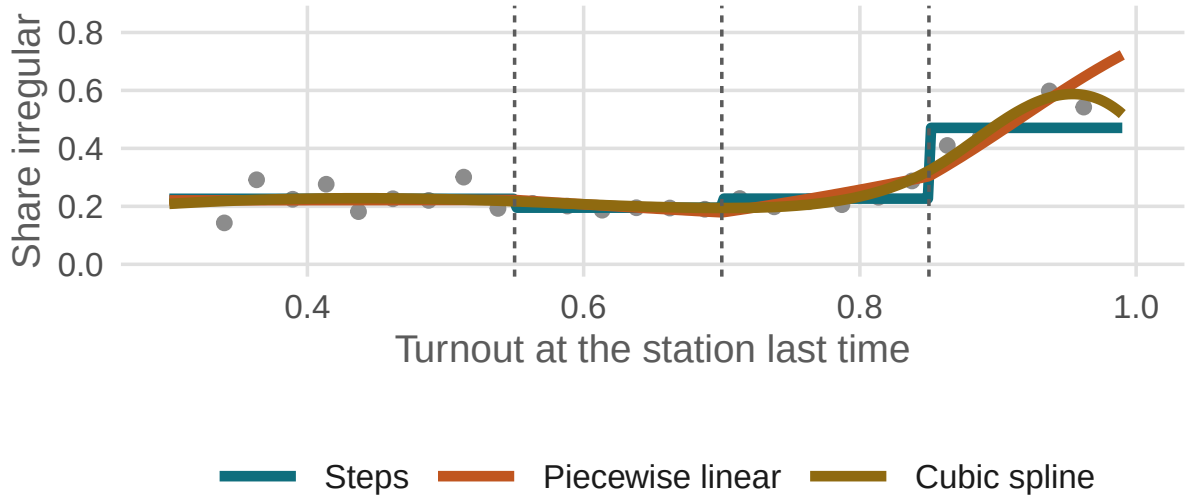
For example, x, x^2, x^3 gives a cubic polynomial, but there are many options.

This is still just a logit, now with $J + 1$ coefficients, estimated the usual way.

The more columns we add, the more flexible the fit. But add too many and we **overfit**: we fit last election's noise as well as its signal. So we face **two choices**: which basis functions to use, and how many of them.

Piecewise polynomials

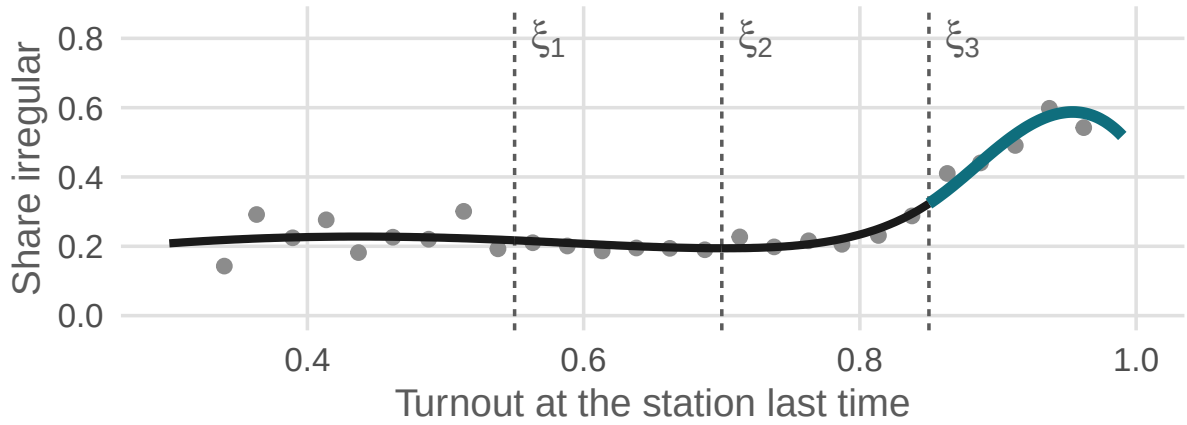
Suppose we cut turnout at three **knots** (dashed). We choose where the pieces start and stop, and the data decide what happens on each:



1. Steps: a constant on each piece, jumping at every knot.
2. Piecewise linear: a line on each piece, joined at the knots.
3. Cubic spline: a cubic on each piece, joined so the slope and curvature match.

Building a cubic spline

Let's propose a cubic in logit π, then add one term at each knot:



$$\text{logit}\,\pi(x) = \begin{cases} \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 & x < \xi_1 \\ \beta_0 + \cdots + \beta_3 x^3 + \theta_1 (x - \xi_1)^3 & \xi_1 \leq x < \xi_2 \\ \beta_0 + \cdots + \beta_3 x^3 + \theta_1 (x - \xi_1)^3 + \theta_2 (x - \xi_2)^3 & \xi_2 \leq x < \xi_3 \\ \beta_0 + \cdots + \beta_3 x^3 + \theta_1 (x - \xi_1)^3 + \theta_2 (x - \xi_2)^3 + \theta_3 (x - \xi_3)^3 & \xi_3 \leq x \end{cases}$$

Read: each knot adds one term, and that term is zero to the left of its knot.

The design matrix

The **design matrix** for this spline:

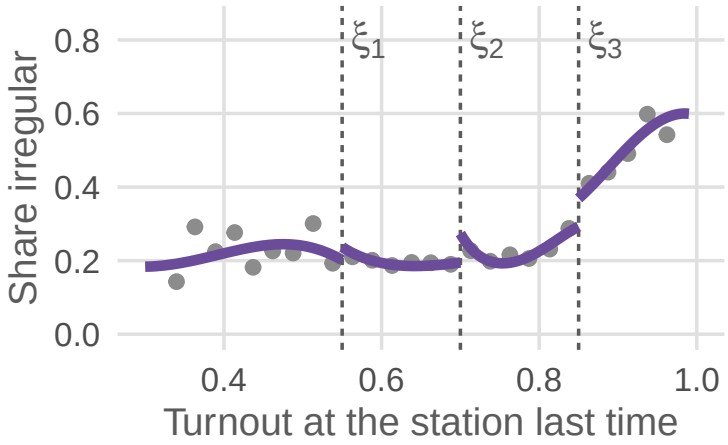
Y	1	x	x^2	x^3	$(x-\xi_1)_+^3$	$(x-\xi_2)_+^3$	$(x-\xi_3)_+^3$
0	1	0.450	0.203	0.091	0	0	0
1	1	0.620	0.384	0.238	0.0003	0	0
0	1	0.780	0.608	0.475	0.0122	0.0005	0
1	1	0.930	0.865	0.804	0.0549	0.0122	0.0005
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

The **+** sets negatives to zero: $(x-\xi_k)_+ = \max(x-\xi_k,\ 0)$. So a knot's column is **0** until turnout passes the knot.

`glm()` fits a logit on this $\boldsymbol{X}$, as on any other: 7 columns, 7 coefficients.

Why the pieces join

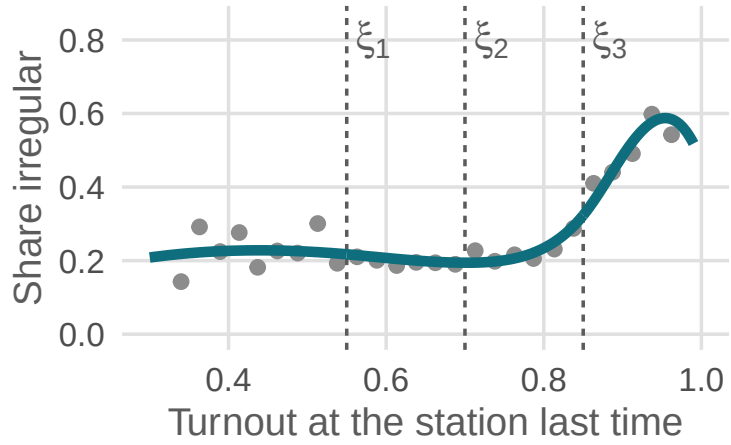
SEPARATE CUBICS



$$\text{logit}\,\pi(x) = \beta_{0k} + \beta_{1k}x + \beta_{2k}x^2 + \beta_{3k}x^3$$

Read: Four cubic models with 16 total coefficients.

CUBIC SPLINE

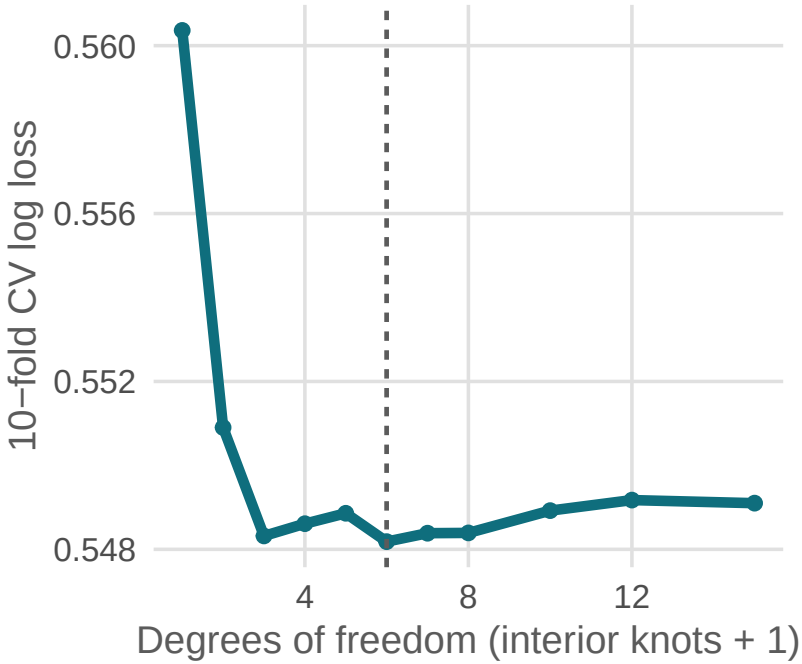


$$\text{logit}\,\pi(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_k \theta_k (x - \xi_k)_+^3$$

Read: One model with 7 total coefficients.

Each spline term starts at zero, so switching it on simply bends the curve without discontinuity.

Splines in practice



A cubic spline misbehaves at the extremities. We typically use a **natural spline** (`ns()`) instead:

- $\beta_0 + \beta_1 x + \sum_k \theta_k (x - \xi_k)_+^3$, with the θ_k constrained
- linear beyond the data (grey)
- interior knots at quantiles of x
- df = interior knots + 1

This makes df (degrees of freedom) our **tuning parameter**! How do we choose it?

Cross validation (CV). In this case the df = 6.

Smoothing splines

Alternatively, put a knot at every unique value of x, and let a **penalty** decide how much $f(x) = \text{logit}\,\pi(x)$ may curve:

$$\hat{f} = \arg\min_f \left\{ \underbrace{-\ell(f)}_{\text{log loss}} + \underbrace{\lambda \int f''(x)^2\,dx}_{\text{penalized total curvature}} \right\}$$

Read: f'' is the curvature at x, how much the line is allowed to curve there.

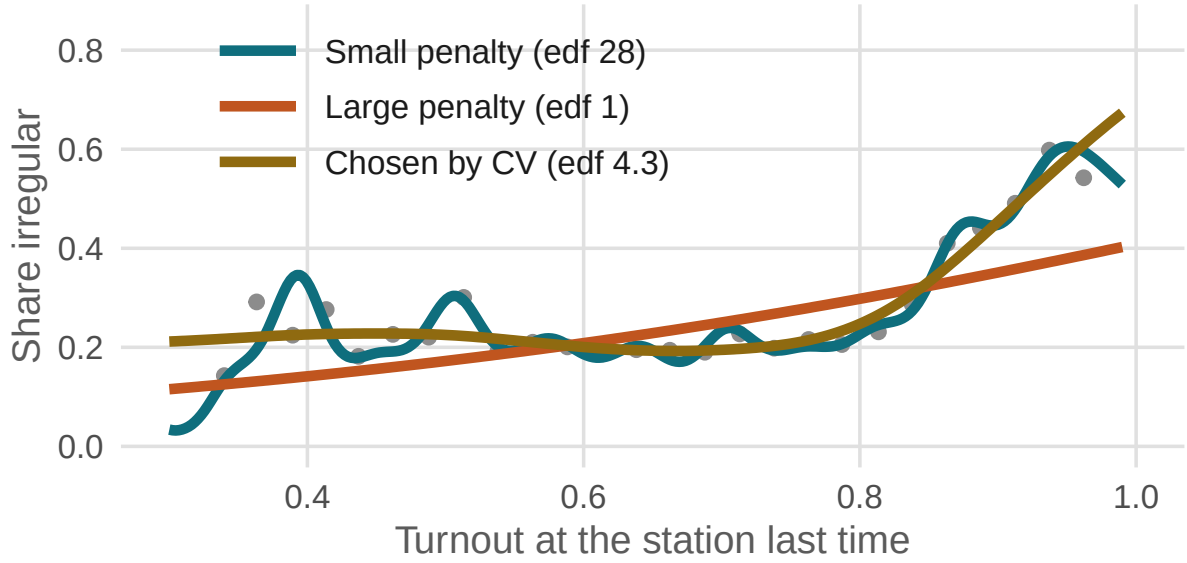
λ is a weight (penalty) on this curvature.

Small λ: log loss dominates, and $\hat{f}$ wiggles. Large λ: curvature dominates, and $\hat{f}$ is a straight line.

λ is another **tuning parameter**.

Smoothing splines

This works very much like ridge regression, but the penalty is on the basis-function coefficients, not the covariates'.



The legend's **effective degrees of freedom** (edf) are df for a penalized curve: how many columns it effectively uses.

Small penalty: wiggly, edf 28. Large penalty: a line, edf 1. CV picks edf = 4.3.

Roadmap

Flexible parametric models

Additive models

Comparing models

Generalized additive models

Now let's use all six features. We give each continuous feature its own smooth function, and add them up on the logit scale:

$$\begin{aligned}\text{logit}\,\pi(\boldsymbol{x}) = \beta_0 &+ f_1(\text{turnout}) + f_2(\text{inc_share}) \\ &+ \beta_3\,\text{urban} + \beta_4\,\text{no_agent} + \beta_5\,\text{flagged} + \text{region}\end{aligned}$$

Read: f_1 and f_2 are smoothing splines, each with its own λ_j. The other features enter as they did in the logit.

This is called a **generalized additive model** (GAM). Additive, because each feature's contribution ignores the others. Note that this is an assumption we make about functional form, not data-driven.

Fitting a GAM

Each f_j is a spline, so it enters as its own basis functions (here 9 each), next to `urban`, `no_agent` and the rest: one design matrix with 27 columns.

Then we fit one penalized logit, with a penalty for each curve:

$$\min \left\{ \underbrace{-\ell}_{\text{log loss}} + \underbrace{\lambda_1 \int f_1''(x)^2\,dx + \lambda_2 \int f_2''(x)^2\,dx}_{\text{penalized total curvature, curve by curve}} \right\}$$

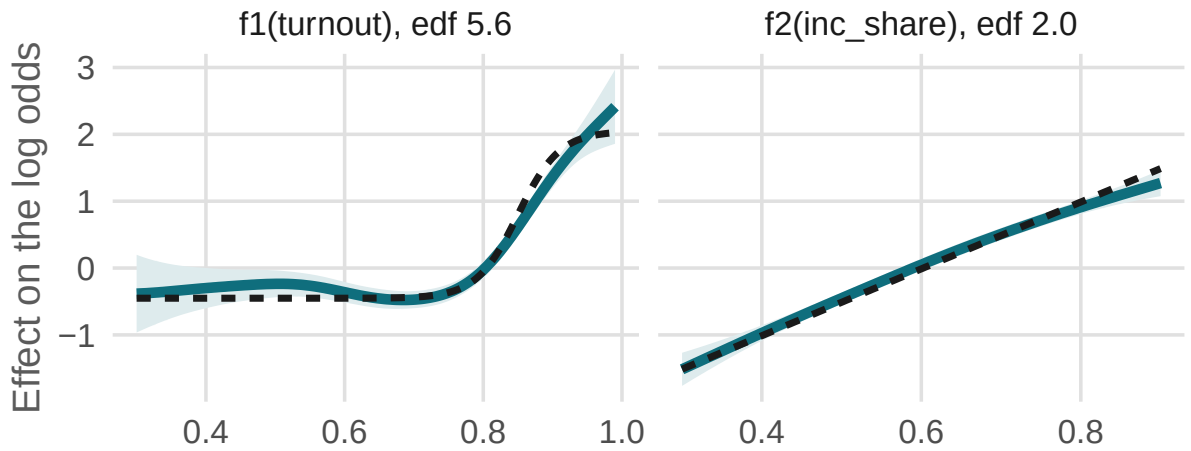
Read: the smoothing-spline objective again, one λ_j per curve.

We now have one tuning parameter per smoothed feature. With many features, CV becomes computationally expensive (e.g. 759,375 combinations for 15 λs over five smoothed covariates).

So we ask `gam()` for **REML** (restricted maximum likelihood): it treats each curve's coefficients like random effects, and estimates all the λ_j at once.

Interpreting a GAM

We typically interpret GAMs visually, showing how logit π changes with each feature, holding the other features fixed:



Each panel is one $\hat{\boldsymbol{f}}_j$ with its edf, and the true $\boldsymbol{f}_j$ dashed. No single $\hat{\beta}$ is the effect of turnout, so we report the curve, with its band.

However, we can summarize a change in $\hat{\pi}$ for a change in $\boldsymbol{x}$: from 80% to 95% turnout, $\hat{\pi}$ rises 34 points on average.

Roadmap

Flexible parametric models

Additive models

Comparing models

Evaluating our candidate classifiers

Model	Test AUC	Turnout effect
Logit	0.825	0.087
Spline logit	0.838	0.349
GAM	0.838	0.344
Random forest	0.858	0.290
Boosting	0.858	0.350
Truth	0.869	0.306

On this election's stations. Turnout effect: the average change in $\hat{\pi}$ from 80% to 95% turnout.

For ranking stations, the forest and boosting still win. Why?

Every split in a tree is an interaction. The GAM has no interactions unless we specify these in the model.

In R

```
fit_ns  <- glm(y ~ urban + no_agent + flagged + region +
                 ns(inc_share, df = 4) + ns(turnout, df = 6),
               data = train, family = binomial)
fit_gam <- gam(y ~ urban + no_agent + flagged + region +
                 s(inc_share) + s(turnout),
               data = train, family = binomial, method = "REML")
```

`ns()` is in `{splines}` and `gam()` in `{mgcv}`.

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **7.3–7.7**. statlearning.com.
- Hainmueller, J. & Hazlett, C. (2014) "Kernel Regularized Least Squares: Reducing Misspecification Bias with a Flexible and Interpretable Machine Learning Approach." *Political Analysis* 22(2): 143–168.